

DriftGate: Combining Personal and Shared Exits for Split Inference on Mobile Devices

Yujin Shin, Jeong Gil Ko, Senior Member, IEEE, and Pei Zhang, Member, IEEE

Abstract—Personalized split learning gives a mobile device two classifiers for each request. A device exit is trained for the user’s own classes, and an edge exit is trained across the users of a cell. When users move, requests alternate between classes the device has learned and classes it has not, and the two exits are accurate on different parts of this traffic. Common split inference offloads a request when the device exit is uncertain, but a personalized exit is also confident on many classes it has not learned, so its entropy separates the two kinds of requests only weakly. We present DriftGate, which answers an offloaded request with a weighted average of the class probabilities of both exits. DriftGate corrects the edge exit for the class mix of its cell, sets the weight from how confident the two exits have been on the device’s recent offloaded requests, and offloads only uncertain requests. We compare it with ten other inference rules on the same trained models in eight settings that include real mobility traces. Over the first simulated day, which includes early training, DriftGate has the highest mean accuracy in all eight settings, 0.1 to 1.9 percentage points above the strongest alternative. The margin is smaller once the models are trained. After round 30, DriftGate ranks first in six of the eight settings with margins of -0.2 to $+1.1$ points, and when trained models are replayed over a day of mobility it is 0.6 points above the strongest alternative on GeoLife traces but 0.3 points below a plain probability average in the commute scenario. Over the first day, DriftGate offloading at most half of the requests is more accurate than every rule that offloads all of them in five of the eight settings. [TODO: One sentence with the measured on-device latency and energy, once measured.]

Index Terms—Split learning, mobile edge computing, personalization, early exit, user mobility, label shift.

1 Introduction

Mobile applications run neural networks on images, speech, and sensor data, but a full model can exceed the compute and memory budget of a phone. Split inference divides the model between the device and a nearby edge server. The device runs the first layers and sends an intermediate feature, and the edge runs the rest of the model [1], [2]. Split learning trains such a model across many users without collecting their raw data [3], [4], [5]. Personalized split learning adds a classifier at the end of the device part [6], [7]. We call each input for which an application needs a class label, such as one camera frame, a request. Each request therefore has two classifiers available. The device exit is personalized to the classes in its user’s training data, and the edge exit is trained across all users of a cell.

Which of the two classifiers gives the right answer depends on where the user is. Consider a camera application that recognizes objects. At home it mostly sees objects that the user has photographed before, and in a shopping district it sees products and signs that other users of that area have photographed. In the commute scenario that we simulate, about one in nine requests of a device at home comes from classes outside its own training data, and about one in two after the user leaves home. The two exits are accurate on different parts of this traffic. Over the first simulated day, the device exit is correct on 86% of requests from the user’s own classes but on only 7% of requests from classes learned

by other users of the current cell. The edge exit is correct on 72% and 41% of the same requests. Neither exit is the better choice for the whole day.

A natural answer is to choose an exit for each request. Common split inference answers on the device when the device exit is confident and otherwise sends the request to the edge [2], [6], [8]. This rule works well when the device exit is uncertain on classes it has not learned. A personalized classifier often is not. It places most of its probability on the user’s own classes even for a request from another class, so its entropy separates the two kinds of requests only weakly (an area under the ROC curve of 0.64). Confidence-based offloading therefore sends 92% of the requests to the edge and is about as accurate as always using the edge exit.

Another answer is to change how strongly each device personalizes during training, for example by keeping more of the local model at home and more of the shared model while away [9]. We tested such a policy with an oracle that knows whether each device is at home. It raised the accuracy of devices at home by 3.7 points and lowered that of devices away from home by 1.7 points. Because the policy changes the personalization of residents and visitors together, this experiment shows a trade-off of the policy but does not isolate its cause. These observations led us to look at inference rules that leave training unchanged.

A device does not need to choose between its two exits. If it averages the class probabilities of both exits, the edge exit’s answer remains large on a request that the device exit has never learned, as long as the edge exit is confident and the device exit spreads its probability over the user’s classes. A product of probabilities, which is what adding logits computes, instead scales the edge probability by the device probability,

-
- Y. Shin and J. G. Ko are with the Intelligent Embedded Systems Lab, Yonsei University, Seoul, Republic of Korea.
 - P. Zhang is with the University of Michigan, Ann Arbor, MI, USA.

so a near-zero device probability can outweigh a confident edge answer. The average still has a problem. The edge exit is trained on the class mix of the whole cell, in which the device's own classes make up about one quarter of the samples, while a device at home receives these classes in about 88% of its requests. The edge exit therefore discounts the classes that dominate the device's traffic at home.

We present DriftGate, an inference method for personalized split learning under user mobility. DriftGate answers with a weighted average of the class probabilities of the two exits. Before averaging, it corrects the edge exit for the class mix of its cell, so that the device's own classes are no longer discounted. It sets the weight of each exit from how confident the two exits have been on the device's recent offloaded requests, without labels. Finally, it sends only the requests on which the device exit is uncertain to the edge, so the device can trade accuracy for communication. DriftGate does not change training and adds a few microseconds of computation per request.

We evaluate DriftGate in eight settings. They include a commute scenario, real mobility traces from the GeoLife dataset [10], faster movement, partial participation, cell-wide changes in the class mix, random mobility, 100 classes, and a ResNet-18 split [11]. We compare it with ten inference rules that cover confidence-based offloading, each single exit, the probability average, the logit sum used by FedRoD [12], the entropy objective of FedTHE [13], the geometric ensembling with early exit of ZTW [14], label-shift correction [15], and a learned per-request weight. Because our scenarios train the models from scratch during one simulated day, we report three evaluation conditions: the whole first day, the rounds after the first hours of training, and a replay of the day with the trained models frozen. Over the first day, DriftGate has the highest mean accuracy in all eight settings, 0.1 to 1.9 points above the strongest alternative and 5.4 to 12.5 points above confidence-based offloading. After round 30, it ranks first in six settings, and the margins over the strongest alternative range from -0.2 to $+1.1$ points. In the replay with trained models, it is 0.6 points above the strongest alternative on GeoLife traces and 0.3 points below the probability average in the commute scenario. **[TODO: Summary of measured on-device latency and energy.]**

This paper makes the following contributions.

- We show how user mobility affects inference in personalized split learning. The device exit and the edge exit are accurate on different requests, and the confidence of the device exit separates these requests only weakly.
- We design DriftGate, which averages the probabilities of the two exits after correcting the class bias of the edge exit, sets the weight of each exit without labels, and supports selective offloading. It runs on top of the trained model and does not change training.
- We evaluate DriftGate against ten inference rules in eight settings, including real mobility traces, separate the effect of early training from that of trained models, and report which parts of the design matter under each condition.

The rest of the paper is organized as follows. Section 2 measures how the two exits and the two common remedies behave under mobility. Section 3 reviews related work. Section 4

defines the system and the inference problem, and Section 5 presents DriftGate. Section 6 reports the evaluation, Section 7 discusses the scope of the method, and Section 8 concludes.

2 Inference under User Mobility

This section measures how personalized split learning answers requests when users move. We use the commute scenario S1 described in Section 6.1. Fifty devices move between four residential cells and a hub cell over a day, the model is trained with a fixed personalization ratio, and accuracy is averaged over the whole day. Because the model is trained from scratch during this day, the first hours also include early training, and we note where results after round 30 differ. We first ask whether the two exits favor different requests, and then test the two common ways of handling such requests.

2.1 Two Exits for One Request

In personalized split learning, the model of a device has a device block and a device exit, and the edge server of each cell has an edge block and an edge exit [6], [7]. The device block maps a request to an intermediate feature. The device exit classifies this feature on the device. If the feature is sent to the edge, the edge block and the edge exit return a second class-probability vector. During training, each device updates its device block with its own labeled data and then mixes it with the average block of its cell. The personalization ratio λ is the share of the locally trained device block that a device keeps in this mix (Section 4.2 gives the update). The edge block is trained on the features of all devices in the cell. The device exit is therefore tied to the classes in its user's data, while the edge exit covers the classes of everyone in the cell.

We call the classes in a device's local training data its own classes and describe a request by its relation to the device that receives it, following the terms of SplitOMC [7]. A request is Main if its class is one of the device's own classes. It is out-of-preference (OOP) if its class belongs only to the own classes of other devices in the current cell, and out-of-region (OOR) otherwise. Our simulation sets how many OOP and OOR requests a device receives per Main request ($\rho = 0.1$ at home and $\rho = 0.8$ elsewhere, Section 6.1). With these settings, 11.5% of the requests of a device in its home cell are OOP or OOR, and this share rises to about 51% after the device leaves its home cell. At 05:00 every device is at home, and during the day only about 35% of them are.

2.2 The Two Exits Favor Different Requests

Does one exit serve a moving device well throughout the day? Fig. 1 answers this question by measuring each exit alone. Panel (a) splits devices by location, and panel (b) splits requests by their relation to the device. The device exit is more accurate at home (77.5% against 69.9%), and the edge exit is more accurate away from home (54.8% against 45.9%). Panel (b) shows the reason. The device exit is correct on 86.3% of Main requests but on only 7.5% of OOP requests and 3.3% of OOR requests. The edge exit is correct on 72.3%, 40.8%, and 22.5% of them. The pattern is the same after round 30 (Main 87.1% against 75.8%, OOP 8.6% against 42.3%). Each exit is better on one kind of request, and a moving device receives both kinds in the same day. A system that uses one exit for all requests therefore loses accuracy either at home or away.

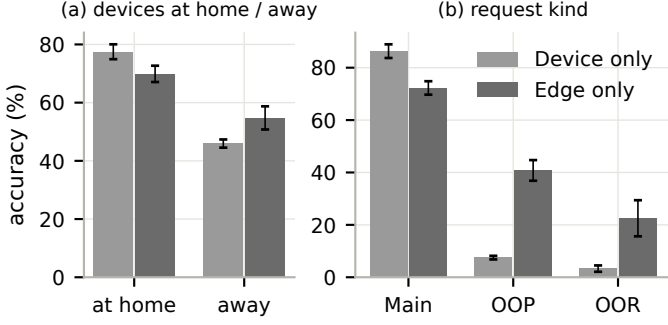


Fig. 1. Accuracy of the device exit alone (Device only) and the edge exit alone (Edge only) in the commute scenario S1 over the first day. (a) Devices in their home cell and devices in another cell. (b) Requests from the device's own classes (Main), from classes learned only by other devices of the current cell (OOP), and from other classes (OOR). Bars show the mean over five seeds and error bars the standard deviation.

2.3 Choosing One Exit per Request

The common way to use both exits is to choose one of them for each request. Split inference systems answer on the device when the device exit is confident and send the request to the edge otherwise [2], [6], [8]. This rule works best when the device exit is uncertain on the requests that it cannot classify. We measure how well the entropy of the device exit, in nats (natural logarithm), separates OOP and OOR requests from Main requests. The area under the receiver operating characteristic curve (AUROC) is 0.64 over the first day, where 0.5 means no separation. A personalized classifier places most of its probability on its user's classes even for a request from another class, a form of the overconfidence that neural classifiers show on inputs outside their training data [16], [17].

The weak separation shows up in accuracy. With an entropy threshold of 0.8 nats, the default of the SplitOMC implementation [7], confidence-based offloading sends 92% of requests to the edge. Over the first day it reaches 62.9%, the same as always using the edge exit (62.9%). After round 30 it again matches the edge exit (64.8% against 64.9%). Choosing the exit with the lower entropy for each request reaches 64.3% over the first day and 65.2% after round 30. A per-request choice needs a per-request signal that tells the two kinds of requests apart, and the confidence of the device exit gives only a weak one.

2.4 Adapting Personalization during Training

The other common remedy changes the model instead of the inference. A device could keep more of its local model when its traffic matches its own data and more of the shared model when it does not [9]. We test this idea with an oracle that knows whether each device is at home. It sets $\lambda = 0.70$ at home and 0.15 away from the first round on, the upper and lower bounds of the range that our adaptive controllers use, while the baseline keeps $\lambda = 0.4$ for every device. Both are evaluated with confidence-based offloading on three development seeds.

Fig. 2 shows the result. The policy raises the accuracy of devices at home by 3.7 points and lowers the accuracy of devices away by 1.7 points, and the whole-day gain is 0.9

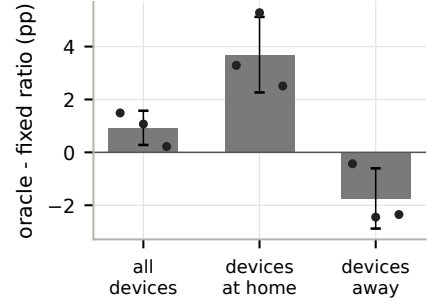


Fig. 2. Accuracy gain of a location-aware training policy that sets each device's personalization ratio from its true location (0.70 at home, 0.15 away) over a fixed ratio of 0.4 in S1, for all devices, devices at home, and devices away. Both use confidence-based offloading at inference. Bars show the mean over three seeds, error bars the standard deviation, and dots the seeds.

points. The policy changes the ratio of a device both at home and away, and a device that keeps more of its local model at home also contributes a more specialized block to the cell average that visitors use. The experiment does not separate these effects, so we report the home gain and the away loss as a trade-off of this policy rather than attribute the loss to residents. Learning the ratio of each device from its own training loss, as APFL does [9], gives 62.9% with the better of two learning rates on seeds 0 to 2, 0.3 points below the fixed ratio on the same seeds.

2.5 Implications

The measurements point to three requirements. First, a device needs both exits within the same day, because each exit is better on one kind of request. Second, the decision should not rest only on the confidence of the device exit for a single request. Third, a remedy that acts at inference changes only the answers of the requesting device, whereas the training-time policy we tested changes the shared model and came with a loss for devices away from home. These observations lead us to keep training unchanged and to combine the two exits at inference time, as Section 5 describes.

3 Related Work

Table 1 summarizes how the inference methods closest to ours use a personalized and a shared classifier.

3.1 Split Inference and Early Exits

Split computing places the first layers of a model on the device and the remaining layers on an edge server or the cloud, and chooses the split point from compute and network conditions [1], [18]. Early-exit networks attach classifiers to intermediate layers and stop when an exit is confident [8], [19]. SPINN combines both ideas and decides at run time whether a request leaves at an on-device exit or continues to the cloud [2]. In personalized split learning, SplitGP trains a personalized device-side model and a shared server-side model and sends a request to the server when the device-side output is uncertain [6]. These systems answer each request with one exit and use the confidence of an earlier exit to make that choice. Section 2.3 shows that, under mobility, the personalized exit is often confident on classes it has never

learned. DriftGate answers offloaded requests with both exits and uses confidence only to decide which requests to offload.

Zero Time Waste (ZTW) reuses the predictions of earlier exits [14]. It passes the logits of each internal classifier to the next one through cascade connections, combines the exits with a weighted geometric mean whose exit weights and class-specific bias terms are trained on the training data, and stops when the largest probability of the combined prediction exceeds a threshold. All exits in ZTW are trained on the same data distribution. The exits of personalized split learning are not, because the device exit has never seen the classes of other devices. A geometric mean multiplies probabilities, so a near-zero device probability on such a class can outweigh a confident edge answer. DriftGate uses an arithmetic average and corrects the class bias of the edge exit before combining. Our evaluation includes the geometric ensemble with early exit of ZTW, without the cascade connections and the trained weights and biases, because the two exits are trained beforehand.

3.2 Personalized Federated and Split Learning

Personalized federated learning balances what a client learns from its own data with what it learns from others. APFL learns a mixture of a local and a global model, Ditto and pFedMe regularize local models toward a shared one, and FedRep shares a representation while keeping a local head [9], [20], [21], [22]. These methods set the balance during training. Section 2.4 shows that, in the location-aware policy we tested, a training-time balance that helped devices at home came with a loss for devices away. Multi-cell split learning considers devices that move between overlapping cells and studies how the class mix of a cell affects training [7].

Kim et al. address label shift in personalized split federated learning by changing training [23]. They pretrain personalized client models together with a large server model, update the client models online by distilling knowledge from the server model when the label distribution shifts, and answer in real time through an early exit of the client model. DriftGate instead leaves both models unchanged and changes only how the outputs of the two exits are combined. Our comparison holds the trained models fixed and varies the inference rule, so it does not cover methods that update the models during deployment. The two approaches could be combined, because DriftGate needs only the outputs of the two exits.

Two methods combine a shared and a personalized classifier at inference time. FedRoD trains a generic head with a class-balanced loss and a personalized head that is added to it, and it predicts with the sum of the two heads' logits [12]. Adding logits is equivalent to multiplying class probabilities. FedTHE ensembles a global and a personalized head with a weight chosen for each test sample [13]. It mixes the logits of the two heads and chooses the weight by minimizing the entropy of the softmax of the mixed logits together with a feature-alignment loss, weighting the two losses by the similarity of the heads' predictions. DriftGate sets one weight per device from the average confidence of the two exits over recent requests, and it corrects the class bias of the shared exit without changing its training. Our evaluation includes the logit sum of FedRoD without its balanced training, and

the entropy objective of FedTHE applied to the logits of the two exits without the feature-alignment term, whose feature descriptors assume one feature space shared by both heads.

3.3 Label Shift and Prior Correction

When the class proportions of the test data differ from those of the training data, the outputs of a classifier can be corrected with Bayes' rule. Saelens et al. estimate the new class proportions with expectation maximization (EM) and rescale the class probabilities [15]. Later work estimates the proportions from a confusion matrix [24] or improves the calibration that the estimate depends on [25]. Logit adjustment applies the same correction with known class priors to train classifiers for long-tailed data [26]. These methods correct one classifier toward an estimate of the current class mix. For a moving device, this mix changes with the device's location, and an estimate computed from the edge exit inherits the edge exit's bias against the device's own classes. DriftGate does not estimate the mix. It rescales the edge exit toward an even split between the device's own classes and all other classes, and lets the personalized device exit supply the device-specific preference. Our evaluation includes EM-based correction of the share of other classes as a baseline.

4 System Model and Problem

4.1 Devices, Cells, and Requests

We consider K mobile devices and L cells. Each cell has an edge server that serves the devices inside its range, and neighboring cells can overlap. At time t , device k belongs to a set Z_k^t of one or two cells, and Z_k^t changes as the device moves. Each device has a home cell where its user spends the night.

The local training data of device k covers a set M_k of its own classes out of C classes. The user issues inference requests over the day, each an input for which the application needs a class label. A request x with class y is a Main request if $y \in M_k$, an out-of-preference (OOP) request if y belongs only to the own classes of other devices in the current cells, and an out-of-region (OOR) request otherwise. Table 2 summarizes the notation. The share of Main requests is high in the home cell and drops when the device visits other cells. The system never observes y , the kind of a request, or this share at inference time. The edge server knows M_k , because in split learning the labels of the training samples are sent to the edge.

4.2 Split Model and Training

The model of device k consists of a device block h_k and a device exit f_k^d . The edge server of cell z holds an edge block and an edge exit, written together as f_z^e . For a request x , the device computes the feature $h_k(x)$ and the probabilities $p^d(\cdot|x) = \text{softmax}(f_k^d(h_k(x)))$. If the feature is sent to the edge, the edge returns $p^e(\cdot|x) = \text{softmax}(f_z^e(h_k(x)))$. A device that belongs to two cells receives the average of the two cells' logits.

Training follows multi-cell personalized split learning [6], [7]. We summarize the procedure used in our implementation. Training proceeds in rounds. In each round, device k trains on labeled samples from its own classes with the loss

$$\mathcal{L}_k = \gamma \text{CE}(f_k^d(h_k(x)), y) + \frac{1-\gamma}{|Z_k^t|} \sum_{z \in Z_k^t} \text{CE}(f_z^e(h_k(x)), y), \quad (1)$$

TABLE 1

How inference methods use a personalized and a shared classifier. “Class adjustment” refers to adjusting a classifier for the class mix of its training data.

Method	Classifiers per request	How the answer is formed	Signal used at inference	Class adjustment
SplitGP [6], SPINN [2]	One exit	Device or server exit	Confidence of the device exit	None
ZTW [14]	Exits up to the stopping point	Weighted geometric mean	Confidence of the combined prediction	Trained class-specific bias
FedRoD [12]	Generic and personalized head	Sum of logits	None	Class-balanced training
FedTHE [13]	Global and personalized head	Weighted logit sum, weight per sample	Entropy of the mixed prediction and feature alignment	None
EM correction [15]	One classifier	Rescaled probabilities	Class mix estimated from recent predictions	Toward the estimated mix
DriftGate	Device and edge exit	Weighted probability average, weight per device	Mean entropy of both exits on recent offloaded requests	Toward an even split of own and other classes, at inference

TABLE 2
Notation.

Symbol	Meaning
k, z, c	Device, cell, and class index
Z_k^t	Cells that device k belongs to at time t
M_k	Own classes of device k
$p^d(\cdot x), p^e(\cdot x)$	Class probabilities of the device and edge exits
π_k	Class mix of the data that trained the edge exit for device k
a_k	Share of M_k in π_k
$\tilde{p}^e(\cdot x)$	Edge probabilities after prior correction
r	Target share of classes outside M_k in the correction
w_k	Weight of the device exit when combining
$\bar{H}_k^d, \bar{H}_k^e$	Mean entropy of the two exits on recent requests
τ, β	Offloading threshold and offloading budget
λ, Λ	Device-to-cell and cell-to-global mixing ratios in training

with $\gamma = 0.5$. After local training, each cell averages the device blocks of its members. This average $\bar{\theta}_z$ and the edge block of the cell are each mixed with their averages over all cells, for example $\bar{\theta}_z \leftarrow \Lambda \bar{\theta}_z + (1 - \Lambda) \bar{\theta}_{\text{all}}$. Each device then installs

$$\theta_k \leftarrow \lambda \theta_k^{\text{local}} + (1 - \lambda) \bar{\theta}_{Z_k}, \quad (2)$$

where θ_k^{local} is its locally trained device block and $\bar{\theta}_{Z_k}$ is the average of its cells’ blocks. A larger λ keeps more of the local model. We use $\lambda = 0.4$ and $\Lambda = 0.5$ throughout. DriftGate does not change this procedure.

Because the edge block of a cell is trained on the samples of all its members, the edge exit learns the class mix of the cell. Let $n_z(c)$ be the number of samples of class c used to train cell z in the current round. We write the class mix that shaped the edge exit used by device k as

$$\pi_k(c) = \frac{1}{|Z_k^t|} \sum_{z \in Z_k^t} \left[\Lambda \frac{n_z(c)}{\sum_{c'} n_z(c')} + (1 - \Lambda) \frac{\sum_{z'} n_{z'}(c)}{\sum_{z', c'} n_{z'}(c')} \right], \quad (3)$$

which follows the cell-to-global mixing of the edge blocks. Averaging model parameters does not average the class priors that the models have learned exactly, so π_k is an approximation of the class mix that shaped the edge exit. We use it because the edge already holds the class counts. A cell without training samples in a round uses the mix of all cells. The share of the device’s own classes in this mix is $a_k = \sum_{c \in M_k} \pi_k(c)$, which we clip to $[0.01, 0.99]$. In the commute scenario, a_k is about 0.25 on average.

4.3 Inference Problem

For each request x of device k , an inference rule makes two decisions. It decides whether to offload, $o(x) \in \{0, 1\}$, that is, whether to send $h_k(x)$ to the edge. It then produces an answer $\hat{y}(x)$ from $p^d(\cdot|x)$ and, if $o(x) = 1$, from $p^e(\cdot|x)$. The rule may use information that the device and the edge already hold, such as recent outputs, M_k , and π_k , but not labels at inference time and not the location of the device.

Let $\mathcal{T}$ be the evaluation times of a day and $\mathcal{K}_t$ the devices that issue requests at time t . The accuracy of a rule over the day is

$$A = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \frac{1}{|\mathcal{K}_t|} \sum_{k \in \mathcal{K}_t} \Pr[\hat{y}(x) = y | k, t]. \quad (4)$$

The goal is to maximize A while the share of offloaded requests stays within a budget β , so that $\mathbb{E}[o(x)] \leq \beta$. When $\beta = 1$, every request can use both exits, and the problem reduces to combining the two probability vectors well.

5 DriftGate

DriftGate changes only how a request is answered. It keeps the trained device and edge models as they are and adds three steps that follow from Section 2. It combines the two exits instead of choosing one, it corrects the edge exit for the class mix of its cell, and it sets the weight of each exit without labels. A fourth step limits offloading to the requests where the edge is most useful. Fig. 3 shows the resulting path of a request.

5.1 Combining the Two Exits

Section 2.2 showed that the device exit is accurate on Main requests and the edge exit on OOP and OOR requests, and Section 2.3 showed that the device exit cannot tell these requests apart by its own confidence. DriftGate therefore uses both exits on every offloaded request and answers with

$$\hat{y}(x) = \arg \max_c w_k p^d(c|x) + (1 - w_k) \tilde{p}^e(c|x), \quad (5)$$

where $\tilde{p}^e$ is the corrected edge output of Section 5.2 and $w_k \in [0, 1]$ is the weight of the device exit.

The form of the combination matters. On a Main request, the device exit usually puts a large probability on the correct

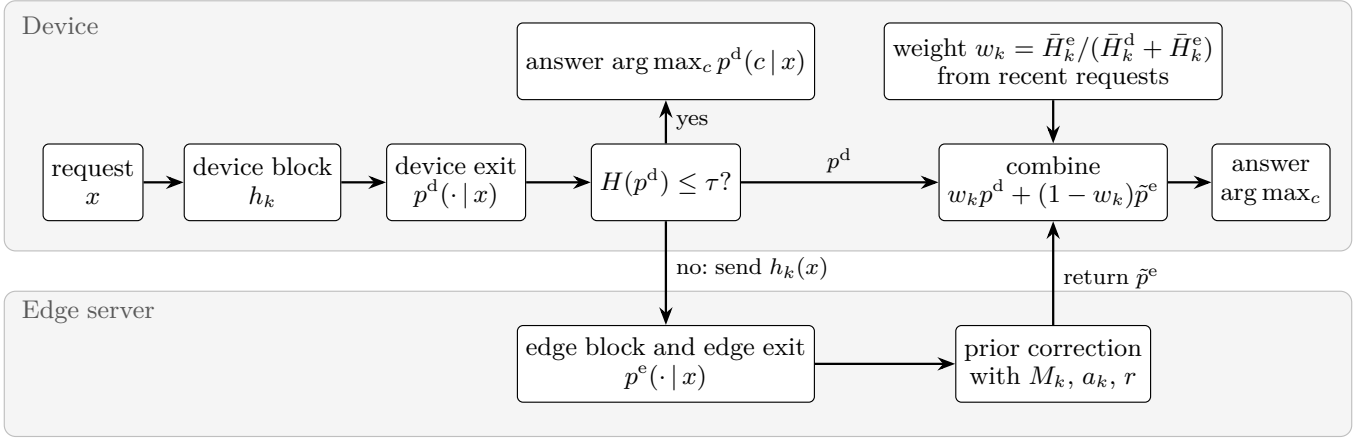


Fig. 3. Inference with DriftGate. The device answers a request locally when its exit is confident. Otherwise it sends the intermediate feature to the edge, which returns class probabilities corrected for the class mix of the cell, and the device answers with a weighted average of both exits. The weight comes from the mean entropy of the two exits on the device’s recent requests.

class, and the average follows it unless the edge exit puts a larger weighted probability on another class. On an OOP request, the device exit has never learned the correct class and spreads its probability over its own classes, while the edge exit may put a large probability on the correct class. The average keeps $(1 - w_k)$ times this probability, so the edge answer wins when this term exceeds the combined probability of every other class, which happens when the edge exit is confident and the device exit spreads its probability. A product of probabilities, which is what adding logits computes [12], [14], scales the edge probability of the correct class by the device probability of that class. When the device probability is close to zero, the product can overrule a confident edge answer. The literature on combining probability forecasts describes the same contrast between linear and logarithmic pooling [27]. Which combination is more accurate depends on the setting in our evaluation (Section 6.6).

5.2 Correcting the Edge Exit for Its Cell

The average in Eq. (5) also depends on how the edge exit weighs the device’s own classes, and by default the edge exit discounts them. A classifier trained on data with class mix π produces probabilities proportional to $p(x | c) \pi(c)$. The edge exit is trained on the class mix π_k of the cell (Eq. (3)), in which the device’s own classes have a share a_k of about 0.25 in the commute scenario. A device at home, however, receives own-class requests about 88% of the time. The edge exit therefore discounts exactly the classes that dominate the device’s traffic, and on a Main request it often prefers a class of another device in the cell.

DriftGate reduces this discount with a rescaling that follows Bayes’ rule [15]. If the class-conditional distributions were the same in training and at inference and the edge probabilities were calibrated, replacing the training mix in this way would give the posterior under the new mix. Neither assumption is guaranteed for the edge exit, so we treat the rescaling as a heuristic and evaluate it empirically. It replaces the training mix π_k with a target mix that keeps the relative frequencies within the device’s own classes and within the

other classes, but gives the other classes a total share r :

$$\tilde{p}^e(c | x) \propto \begin{cases} p^e(c | x) \frac{1 - r}{a_k}, & c \in M_k, \\ p^e(c | x) \frac{r}{1 - a_k}, & c \notin M_k, \end{cases} \quad (6)$$

normalized to sum to one. We set $r = 1/2$, so the corrected edge exit treats the device’s own classes and all other classes as equally likely before seeing the request.

We use this even split instead of an estimate of the device’s current mix for three reasons. First, the true share changes with the device’s location, from about 0.115 at home to about 0.51 away in the commute scenario, and the edge cannot observe it. Second, an estimate computed from the edge exit inherits the bias that the correction is meant to remove. In our development runs (three seeds that we do not use for evaluation, rounds 30 and later), the EM estimate of Saelens et al. [15] gave 0.23 for devices away from home, where the true share was 0.51. Third, the combined answer does not need the edge exit to know the device’s preference for its own classes, because the device exit already carries that preference. The edge exit only needs to stop discounting those classes. The correction uses only M_k and π_k , which the edge already holds from training.

5.3 Setting the Weight without Labels

The weight w_k decides how much the device exit counts in Eq. (5). Its best value depends on the model and the split point. With a small device block, the edge exit is the stronger classifier on most requests. With a large device block, such as the first half of a ResNet-18, the device exit can be stronger than the edge exit overall. A weight tuned for one model can be far from the best for another (Section 6.6), and labels are not available to tune it after deployment.

DriftGate sets the weight from how confident the two exits are on the device’s recent traffic. Let $\bar{H}_k^d$ and $\bar{H}_k^e$ be the mean entropies of the device exit and the edge exit over a window of recent offloaded requests of device k , not including the current request. DriftGate uses

$$w_k = \frac{\bar{H}_k^e}{\bar{H}_k^d + \bar{H}_k^e}, \quad (7)$$

Algorithm 1 DriftGate inference for a request x of device k

Require: threshold τ , window of entropies of earlier offloaded requests of device k ; edge holds M_k , π_k , and r

- 1: compute $h_k(x)$ and $p^d(\cdot|x)$ on the device
- 2: if $H(p^d(\cdot|x)) \leq \tau$ then
- 3: return $\arg \max_c p^d(c|x)$
- 4: end if
- 5: send $h_k(x)$ to the edge server
- 6: edge: compute $p^e(\cdot|x)$, $H(p^e(\cdot|x))$, and $\tilde{p}^e(\cdot|x)$ by Eq. (6); return $\tilde{p}^e$ and the entropy
- 7: $w_k \leftarrow \tilde{H}_k^e / (\tilde{H}_k^d + \tilde{H}_k^e)$ from the window, or $1/2$ if no window exists yet
- 8: $\hat{y} \leftarrow \arg \max_c w_k p^d(c|x) + (1 - w_k) \tilde{p}^e(c|x)$
- 9: add $H(p^d)$ and $H(p^e)$ of this request to the window
- 10: return $\hat{y}$

so the exit that has been more confident on this device's requests receives the larger weight. Section 2.3 showed that the confidence of the device exit on one request is a weak signal. The mean over many requests answers a different question. It does not try to identify the kind of each request, but measures which exit is generally sharper for this device, model, and split point. In our implementation, the window holds the device's earlier offloaded requests of the current evaluation period once there are at least eight of them, and otherwise the offloaded requests of the device's previous period. Before any such requests exist, DriftGate uses $w_k = 1/2$. In the commute scenario, w_k is 0.42 on average with a standard deviation of 0.04 within a run, and with ResNet-18 it is 0.59. The weight thus settles near a value set by the model and split point more than it follows individual requests.

5.4 Offloading Only Uncertain Requests

Combining requires the edge output, so every request that DriftGate combines must be offloaded. When the network or the edge is a bottleneck, the device can answer some requests alone. DriftGate keeps the confidence of the device exit for this purpose. A request with $H(p^d(\cdot|x)) \leq \tau$ is answered with the device exit, and the other requests are offloaded and combined. The threshold τ follows the offloading budget β , for example as the $(1 - \beta)$ quantile of recent device-exit entropies. The confidence of the device exit is too weak to choose the exit that answers a request, but it is useful for ranking which requests gain least from the edge. Requests with very low device-exit entropy are more often Main requests that the device exit already answers correctly, so they gain least from the edge. Since only offloaded requests have an edge output, both $\tilde{H}_k^d$ and $\tilde{H}_k^e$ are computed over the same offloaded requests.

5.5 Deployment and Cost

Algorithm 1 summarizes DriftGate. The edge applies the correction because it already holds M_k and the class counts of its training data. Instead of a single label, it returns C corrected probabilities and one entropy value, an application payload of 44 bytes for ten classes in single precision. For a device in two cells, both edges compute their logits and the average of the two is corrected (Section 4.2), so such a request takes two edge calls; in S1 this adds 0.13 calls per offloaded request. The device keeps two running sums for the weight

and computes one weighted average of two C -dimensional vectors per offloaded request. With NumPy on one core of a server CPU (AMD Ryzen Threadripper PRO 7965WX), the correction, the weighted average, and the argmax take $3.2 \mu\text{s}$ per request when 20,000 requests are processed one at a time. Training, the models, and the uplink payload stay unchanged.

6 Evaluation

We ask five questions. Is DriftGate more accurate than other inference rules under mobility (Section 6.2)? How much of this comes from the early phase of training, and what happens with trained models (Section 6.3)? Which devices, requests, and times of day does the gain come from (Section 6.4)? How much accuracy does it keep when fewer requests are offloaded (Section 6.5)? Which parts of the design matter (Section 6.6)? Sections 6.7 and 6.8 cover larger deployments, the least accurate devices, and the cost on devices.

6.1 Setup

Settings. Table 3 lists the settings. The commute scenario S1 places five cells with a 1 km edge range, four residential cells and one hub, and 50 devices whose users walk from home to the hub and back over a day from 05:00 to 20:00. Training runs in 150 rounds of six minutes. Every device is at home at 05:00, and during the day about 35% are, while the hub holds up to 32 devices. About 14% of the devices are inside two cells at a time. S2 replaces the synthetic paths with 50 weekday user-days from the GeoLife trajectories [10] and places five edges by k -means over the traces. The paths are real, and the recognition requests are generated as in S1. S2 has a weaker commute pattern, and about 60% of its devices stay at home during the day. S1-fast moves the S1 users at vehicle speed, and partial participation lets a random half of the devices train in each round. Two further settings change the class mix of all devices together. In the stepwise change, cell membership is fixed while the share parameter ρ defined below rises from 0 to 0.8 and returns to 0 in five phases over 150 rounds. In random mobility, devices move between cells by a Gauss–Markov model [28] for 120 rounds, and ρ changes from 0 to 0.8 halfway through the run. CIFAR-100 repeats S1 with 100 classes, and ResNet-18 repeats the stepwise change with 16 devices and a ResNet-18 [11] split in the middle. Section 6.7 repeats the S1 layout to reach 200 and 500 devices.

Data and requests. Each device holds two own classes of CIFAR-10 [29], or 20 of CIFAR-100. In S1, a device holds 620 to 2,857 training images. The requests of a device in an evaluation round are test images that are not used by any other part of the system. For every Main request, a device receives on average ρ OOP requests and 0.3ρ OOR requests, following the proportions of SplitOMC [7]. In the mobility settings, $\rho = 0.1$ in the device's home cell and $\rho = 0.8$ elsewhere, so OOP and OOR requests make up about 11.5% and 51% of the traffic. These values are simulation settings, not measurements of a real application. The arrival order of the requests within a round is random and is the same for every inference rule.

Models and training. The CNN has a device block of 0.13 M parameters, a device exit of 0.15 M parameters, and an edge block and exit of 1.38 M parameters. Its intermediate feature has $128 \times 8 \times 8$ values (32 KB in single precision).

TABLE 3
Evaluation settings. Settings other than ResNet-18 and the scale study use 50 devices and five cells.

Setting	Movement and class mix	Data, model	Devices	Seeds
S1 (commute)	Walking commute between four residential cells and a hub	CIFAR-10, CNN	50	5
S2 (GeoLife)	50 user-days of GeoLife traces	CIFAR-10, CNN	50	3
S1-fast	S1 at vehicle speed	CIFAR-10, CNN	50	3
Partial participation	S1, half of the devices train per round	CIFAR-10, CNN	50	3
Stepwise change	Fixed cells, ρ : 0, 0.4, 0.8, 0.4, 0 for all devices	CIFAR-10, CNN	50	5
Random mobility	Gauss-Markov movement, ρ : 0 then 0.8, 120 rounds	CIFAR-10, CNN	50	3
CIFAR-100	S1 with 100 classes	CIFAR-100, CNN	50	3
ResNet-18	Stepwise change	CIFAR-10, ResNet-18	16	3
Scale	S1 layout repeated	CIFAR-10, CNN	200, 500	3

All models are trained from random initialization with SGD (learning rate 0.01, weight decay 10^{-4} , batch size 32, three local epochs per round) and the procedure of Section 4.2 with $\lambda = 0.4$ and $\Lambda = 0.5$.

Inference rules. All rules answer the same requests with the same trained models. Training is the same for every rule.

- Confidence-based offloading, the inference rule of SplitGP [6], answers with the device exit if its entropy is at most 0.8 nats and otherwise with the edge exit.
- Device only and Edge only always use one exit.
- Probability average answers with the mean of the two probability vectors.
- Logit sum adds the logits of the two exits, which is the combination of FedRoD [12]; FedRoD’s balanced training of the generic head is not used.
- Lower-entropy exit uses, for each request, the exit with the lower entropy.
- Logit-entropy weighting applies the entropy objective of FedTHE [13] to each request. It mixes the logits of the two exits with the weight in $\{0, 0.01, \dots, 1\}$ that minimizes the entropy of the softmax of the mixture, and answers with that mixture. FedTHE’s feature-alignment term and test-time history are not used.
- Geometric ensemble with early exit answers with the device exit when its largest probability exceeds a threshold and otherwise with the geometric mean of both exits, following the ensembling and early exit of ZTW [14] without its cascade connections and trained weights. The threshold gives the same offloading ratio as confidence-based offloading.
- Label-shift EM estimates the share of requests outside M_k by EM [15] on the device’s earlier requests and answers with the edge exit corrected to that share.
- Learned weight combines the exits with a per-request weight from a logistic regression that predicts whether a request is Main from nine label-free features, such as the confidence of each exit and their agreement. The regression is trained with labels on development runs, which gives per-request weighting an advantage that a deployed system would not have. It uses the uncorrected edge exit.

DriftGate uses $r = 1/2$ and the window of Section 5.3, where an evaluation period is one evaluation round. Except in Section 6.5, DriftGate and all combining rules offload every request, and the geometric ensemble with early exit offloads the same share as confidence-based offloading. The value of r ,

the minimum of eight requests in the window, and the logistic regression were fixed on development runs whose seeds are not used in this section. The threshold of 0.8 nats is the default of the SplitOMC implementation.

Evaluation conditions and metric. We report the accuracy of Eq. (4). Evaluation takes place every five rounds in the mobility settings and every ten rounds in the stepwise change and random mobility. Because training starts at the beginning of the simulated day, the whole run includes the early phase of training. We therefore report three conditions. The first day averages all evaluation rounds of the run. Round > 30 and round > 50 average the evaluation rounds after rounds 30 and 50. The frozen replay restores the models of S1 and S2 at the end of the day and replays the same day of movement and requests with all parameters, including batch-normalization statistics, frozen. For the replay, we retrained the S1 and S2 runs with the same settings and seeds and kept their end-of-day state, and the edge exit uses the class counts of the last training round for its correction. We report the mean over seeds, and differences are computed per seed before averaging. The strongest alternative in a setting is the rule other than DriftGate with the highest mean accuracy.

6.2 Overall Accuracy over the First Day

Is DriftGate more accurate than other ways of answering requests? Table 4 compares eleven rules in the eight settings over the first day. Rules that choose one exit per request, namely confidence-based offloading and the lower-entropy exit, do not rise clearly above the better single exit. Logit-entropy weighting behaves almost exactly like the lower-entropy exit. Its entropy-minimizing weight is 0 or 1 for 94.6% to 98.7% of the requests in the CNN settings, so it also chooses one exit for most requests. Rules that combine the exits, namely the probability average, the logit sum, and the geometric ensemble, are 2 to 11 points above confidence-based offloading. DriftGate has the highest mean accuracy in all eight settings over the first day. Its gain over confidence-based offloading ranges from 5.4 to 12.5 points, and its gain over the strongest alternative from 0.1 to 1.9 points. With ResNet-18, the mean gain of 0.10 points is smaller than its seed-to-seed standard deviation of 0.61 points, so DriftGate and the strongest alternative cannot be told apart there. In CIFAR-100, where each device owns 20 of 100 classes, the strongest alternative is Label-shift EM, which also corrects the edge exit.

TABLE 4

Accuracy (%) over the first day, which includes early training. The last rows give the gain of DriftGate over confidence-based offloading, its gain over the strongest alternative in each setting, and that rule. Gains are the mean and standard deviation of the per-seed difference.

Rule	S1	S2	S1-fast	Partial part.	Stepwise	Random mob.	CIFAR-100	ResNet-18
Confidence-based offloading	62.87	65.18	63.33	56.45	65.17	58.56	35.84	59.21
Device only	64.16	69.04	64.39	61.75	68.93	67.71	33.93	66.39
Edge only	62.91	65.24	63.38	56.44	65.20	58.61	35.84	57.04
Probability average	67.32	69.88	68.06	62.64	71.44	69.58	38.15	66.39
Logit sum	67.37	70.07	68.12	62.94	71.39	69.23	39.83	66.00
Lower-entropy exit	64.35	66.65	64.90	58.10	68.42	64.22	35.89	66.54
Logit-entropy weighting	64.35	66.65	64.91	58.10	68.42	64.22	35.90	66.54
Geometric ensemble with early exit	67.36	70.06	68.11	62.94	71.38	69.22	39.83	65.99
Label-shift EM	64.84	68.63	65.41	60.45	67.59	65.42	40.44	61.73
Learned weight	66.81	68.59	67.52	62.02	70.69	68.55	37.54	63.61
DriftGate	68.31	71.70	68.96	64.82	72.36	71.09	41.29	66.64
Gain over confidence-based	5.45 (0.47)	6.52 (2.77)	5.63 (0.38)	8.37 (0.15)	7.18 (0.77)	12.53 (1.08)	5.45 (0.67)	7.43 (2.17)
Gain over strongest alternative	0.95 (0.57)	1.63 (1.04)	0.84 (0.25)	1.88 (0.59)	0.92 (0.26)	1.52 (0.53)	0.85 (0.31)	0.10 (0.61)
Strongest alternative	Logit sum	Logit sum	Logit sum	Logit sum	Prob. average	Prob. average	Label-shift EM	Logit-entropy w.

6.3 Early Training and Trained Models

How much of the gain comes from the first hours, when the models are still being trained? We first split the first-day gain by time of day, weighting each period by its share of the evaluation rounds as in Eq. (4). In S1, the six evaluation rounds before the commute, out of 31, contribute 3.4 of the 5.5 points that DriftGate gains over confidence-based offloading and 0.58 of the 0.95 points it gains over the strongest alternative. In these rounds the edge exit is still in its first rounds of training. Table 5 then drops the early rounds. After round 30, DriftGate remains the most accurate rule in six of the eight settings. Its margin over the strongest alternative falls to between -0.21 and $+1.14$ points, and in the stepwise change it is 0.02 points behind the probability average. After round 50, it remains first in five settings. In random mobility, where about half of every device's requests are OOP or OOR after round 60, it falls 1.8 points below the learned weight and below confidence-based offloading. Across conditions, its lead is largest in S2, partial participation, and CIFAR-100.

The windows still use models that change during the day. Table 6 and Fig. 4 therefore replay the same day with the trained models frozen. The retrained first day matches the original runs within 0.05 points for DriftGate. With trained models, every rule is more accurate, and the edge exit gains most, because its early rounds no longer weigh on the day. Confidence-based offloading rises by 8.6 points in S1 and 9.5 points in S2, and DriftGate by 3.6 and 4.8 points. In S2, DriftGate remains the most accurate rule, 0.64 points above the probability average and 1.77 points above confidence-based offloading. In S1, it is 0.33 points below the probability average and ranks fifth, although it is still 0.48 points above confidence-based offloading. By time of day, DriftGate is about one point above the probability average in S1 while devices are at home and up to 1.1 points below it during the commute and the day, when most devices are away. The first-day results of Table 4 therefore include a large contribution of early training. With trained models, the advantage of DriftGate over the other combining rules depends on the scenario.

6.4 Where the Gain Comes From

Which devices benefit from DriftGate? Fig. 5 places each rule by the accuracy of devices at home (horizontal axis) and away

(vertical axis). The single exits sit at opposite corners. The device exit is accurate at home and poor away, and the edge exit and confidence-based offloading are the reverse. DriftGate lies to the right of every other rule. In S1, devices at home reach 79.8%, which is 2.3 points above the device exit alone and 2.5 points above the logit sum. Devices away reach 53.2%, 1.6 points below the edge exit alone. In S2, devices at home are 2.0 points above the device exit alone, and devices away are 0.4 points below the edge exit alone. In the S1 replay, the same trade appears with trained models: devices at home reach 83.5% against 82.5% for the probability average, and devices away 56.5% against 58.5%.

Table 7 shows the same trade by request kind. On Main requests, DriftGate is as accurate as the device exit alone (86.9% against 86.3% in S1), whereas the probability average and the logit sum are 3 to 4 points lower there. On OOP and OOR requests, DriftGate is well above the device exit but below the edge exit and the other combining rules. Main requests make up about two thirds of a device's requests over the day, so in these settings the gain on Main requests outweighs the loss on the others. The accuracy of devices away and the accuracy on OOP and OOR requests measure different things: devices away also receive Main requests, on which DriftGate is accurate.

When during the day does the gain appear? Fig. 6 follows the accuracy of four rules through the first day in S1. Before the commute, almost every device is at home and DriftGate follows the device exit (72.9% against 73.2%). In these early hours the edge exit is still in its first rounds of training, and confidence-based offloading, which sends most requests to it, stays at 55.3%. During the day, when most devices are away, the device exit falls below the edge exit, and DriftGate stays above both exits from the commute until the evening. Its lead over the better single exit is 4.0 points during the commute, 1.7 points during the day, 1.5 points on the way back, and 3.4 points in the evening. Section 6.3 shows how much of the morning lead comes from early training.

6.5 Accuracy and Offloading

How much accuracy does DriftGate keep when fewer requests can be offloaded? Fig. 7 varies the offloading threshold of each rule and plots accuracy against the share of requests sent to the edge. All curves start from the device exit alone at an offloading ratio of zero. For confidence-based offloading,

TABLE 5

Gain (points) of DriftGate over the strongest alternative of each condition, its rank among the eleven rules, and its gain over confidence-based offloading, for the first day and for evaluation rounds after rounds 30 and 50. Gains are means of per-seed differences.

Setting	Over strongest alternative (rank)			Over confidence-based		
	First day	Round > 30	Round > 50	First day	Round > 30	Round > 50
S1	0.95 (1)	0.36 (1)	0.22 (1)	5.45	2.31	1.74
S2	1.63 (1)	1.14 (1)	1.09 (1)	6.52	3.92	3.28
S1-fast	0.84 (1)	0.18 (1)	0.03 (1)	5.63	2.26	1.64
Partial part.	1.88 (1)	0.93 (1)	0.82 (1)	8.37	5.49	4.80
Stepwise	0.92 (1)	-0.02 (2)	-0.02 (2)	7.18	1.21	0.93
Random mob.	1.52 (1)	0.11 (1)	-1.82 (9)	12.53	3.70	-1.07
CIFAR-100	0.85 (1)	0.52 (1)	0.28 (1)	5.45	5.03	4.96
ResNet-18	0.10 (1)	-0.21 (6)	-0.27 (6)	7.43	3.94	3.61

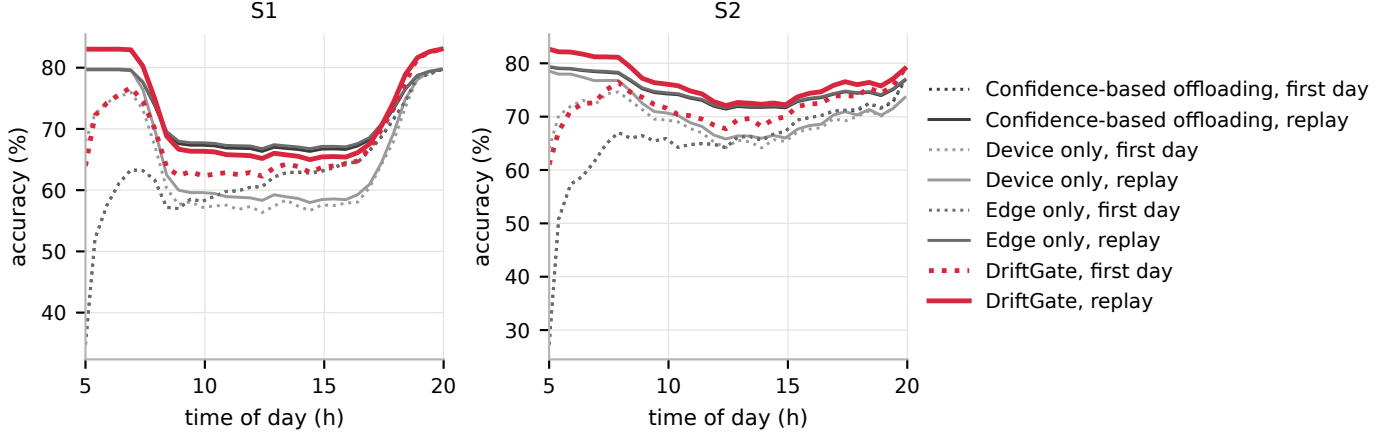


Fig. 4. Accuracy over the day in S1 (left) and S2 (right) on the first day (dotted) and in the frozen replay of the same day with the end-of-day models (solid), for confidence-based offloading, the device exit alone, the edge exit alone, and DriftGate. Seed means.

TABLE 6

Accuracy (%) of the first day and of the frozen replay of the same day with the end-of-day models, in S1 (five seeds) and S2 (three seeds). Geometric ensemble denotes the geometric ensemble with early exit. The last rows give the mean and standard deviation of DriftGate's per-seed gain over confidence-based offloading and over the strongest alternative.

Rule	S1		S2	
	First day	Replay	First day	Replay
Confidence-based	62.86	71.47	65.24	74.76
Device only	64.17	66.38	69.06	71.12
Edge only	62.91	71.74	65.30	74.94
Probability average	67.32	72.27	69.93	75.90
Logit sum	67.37	72.08	70.12	75.86
Lower-entropy exit	64.35	71.71	66.71	75.08
Logit-entropy w.	64.35	71.72	66.71	75.09
Geometric ensemble	67.37	72.03	70.11	75.82
Label-shift EM	64.84	70.17	68.66	74.77
Learned weight	66.82	72.13	68.63	74.53
DriftGate	68.32	71.95	71.74	76.54
vs. conf.-based	5.46 (0.46)	0.48 (0.59)	6.50 (2.79)	1.77 (1.51)
vs. strongest	0.95 (0.57)	-0.33 (0.31)	1.62 (1.04)	0.64 (0.81)

sending more requests to the edge lowers accuracy over the first day, because the additional requests are more often Main requests, which the device exit answers better than the edge exit. The geometric ensemble and DriftGate improve as they offload more, since both use the device exit's output when they combine. Over the first day, DriftGate is above the geometric ensemble at every offloading ratio of 0.3 or more in

TABLE 7

Accuracy (%) by request kind in S1 and S2 over the first day.

Setting	Rule	Main	OOP	OOR
S1	Confidence-based	72.31	40.41	22.37
	Device only	86.31	7.49	3.30
	Edge only	72.27	40.79	22.52
	Probability average	82.36	29.34	15.45
	Logit sum	83.01	27.72	14.76
	DriftGate	86.94	21.63	10.68
S2	Confidence-based	73.46	46.39	8.72
	Device only	84.65	15.29	1.69
	Edge only	73.42	46.92	8.82
	Probability average	80.83	38.50	6.18
	Logit sum	81.32	37.11	5.85
	DriftGate	84.70	30.63	4.86

all eight settings. After round 30, it holds only in S2, partial participation, and random mobility.

Over the first day, DriftGate can also be more accurate than every rule that offloads all requests while offloading at most half of them. Among the thresholds we evaluated, this holds in five settings: S2 (from an offloading ratio of 0.24), partial participation (0.14), stepwise change (0.47), random mobility (0.37), and ResNet-18 (0.23). In S1 and S1-fast, the first such threshold offloads 52% of the requests, and in CIFAR-100 95%, because with 100 classes the device exit is weak on its own. After round 30, it holds only in S2 and partial participation. In the S1 replay, the DriftGate curve

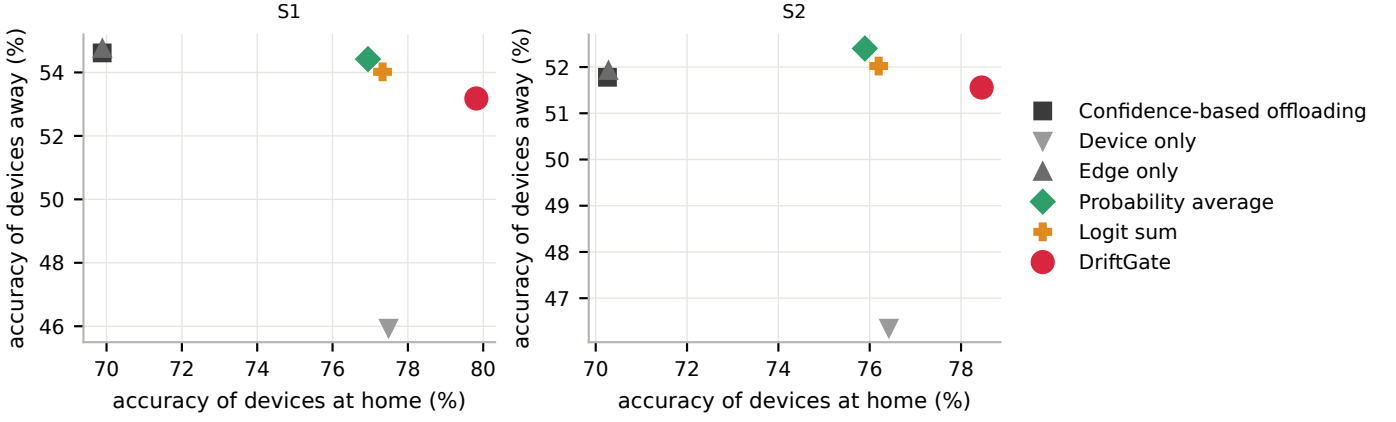


Fig. 5. Accuracy of devices in their home cell against accuracy of devices in another cell for six inference rules in S1 (left, five seeds) and S2 (right, three seeds), over the first day.

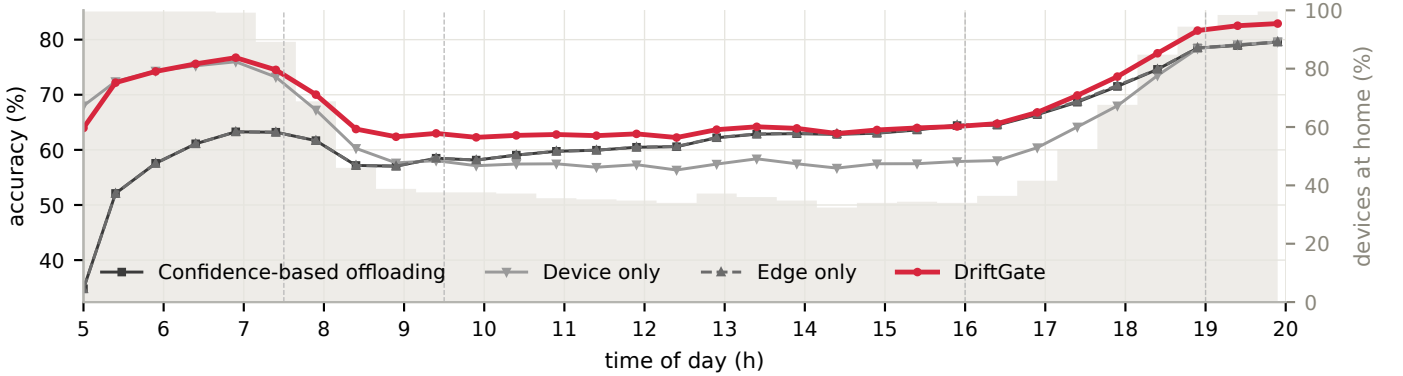


Fig. 6. Accuracy over the first day in S1 for confidence-based offloading, the device exit alone, the edge exit alone, and DriftGate (seed means of five seeds). The shaded background shows the share of devices in their home cell (right axis). Dashed lines separate the pre-commute, commute, daytime, return, and evening periods.

does not reach the probability average at any offloading ratio, and in the S2 replay it does so from 0.51.

The threshold sweep fixes one threshold for the whole day. A deployed device instead needs a threshold that meets a budget online. We therefore also run a simple controller that sets the threshold of each device to the median device-exit entropy of its previous 128 requests, which targets $\beta = 0.5$. It reaches an offloading ratio of 0.50 in every run. With the same offloading decisions, DriftGate is the most accurate way to answer the offloaded requests over the first day in S1 (67.7%, against 67.1% for the logit sum and 64.7% for the edge exit) and S2 (71.8% against 70.6% and 68.2%) and in the S2 replay (76.0% against 75.5% for the probability average). In the S1 replay, it is 0.2 points below the probability average.

6.6 Ablation

Which parts of DriftGate matter? Table 8 changes one choice at a time. Over the first day, removing the prior correction costs the most. With the adaptive weight kept, accuracy drops by 1.2 points in S1, 2.4 points in S2, 1.8 points in random mobility, and 3.8 points in CIFAR-100, but only 0.2 points with ResNet-18. In the S1 replay with trained models, removing the correction instead raises accuracy by 0.5 points. The correction therefore helps most when the edge exit is weak or the class mix is skewed, and it is not helpful in every condition.

The adaptive weight is close to the better fixed weight in each setting rather than better than both. Over the first day, a fixed weight of 0.5 with the correction is within 0.2 points of DriftGate in all five settings. The weight of 0.2 chosen on development runs with the CNN is 2.6 points below DriftGate with ResNet-18, where the device exit is the stronger classifier, but it is 0.6 and 0.1 points above DriftGate in the S1 and S2 replays. In the S1 replay, the corrected edge exit alone is 0.8 points above DriftGate, so with trained models in this scenario the device exit adds little to the corrected edge exit. Refitting the learned weight with the corrected edge exit raises it but leaves it 0.4 to 2.3 points below DriftGate over the first day. Replacing the average with a product, with the correction in place, lowers accuracy in S1, random mobility, ResNet-18, and both replays, and raises it in CIFAR-100 (by 0.4 points) and S2 (by 0.1 points).

The other parameters matter less. Fixed-length windows of 8, 32, and 128 offloaded requests that continue across evaluation rounds change accuracy by at most 0.02 points, because the weight varies little within a run (a standard deviation of about 0.04). Changing the target share r from 0.5 to 0.3 or 0.7 lowers first-day accuracy by up to 2.4 points ($r = 0.7$ in CIFAR-100), although $r = 0.3$ is 0.3 points better in S2 and $r = 0.7$ is 0.2 points better in the S1 replay. These results show that $r = 1/2$ is a reasonable default. They do not show that the edge probabilities satisfy the assumptions

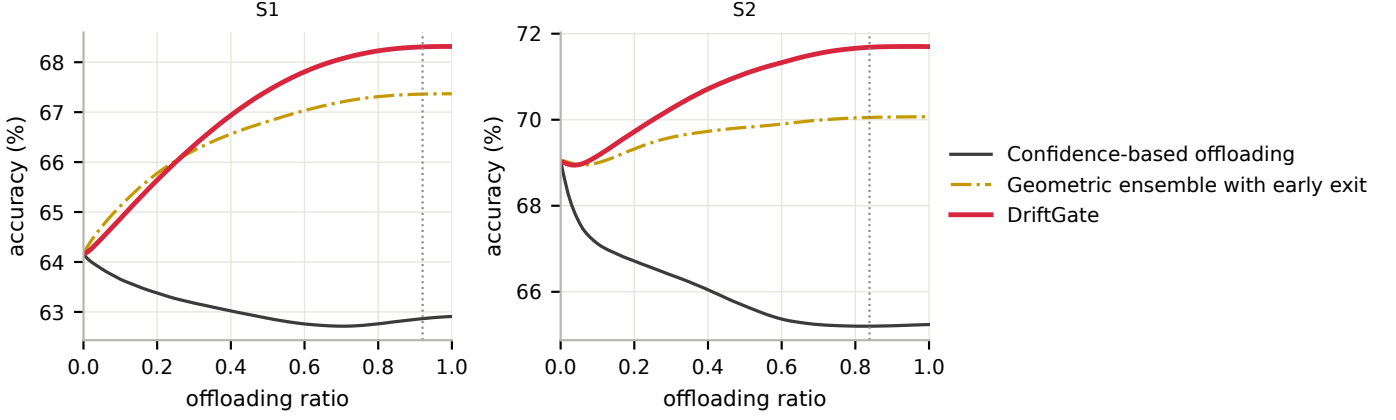


Fig. 7. Accuracy over the first day against the offloading ratio when only requests on which the device exit is uncertain are sent to the edge, in S1 (left) and S2 (right). Confidence-based offloading and DriftGate offload requests whose device-exit entropy exceeds a threshold, and the geometric ensemble offloads requests whose largest device-exit probability is below a threshold. Confidence-based offloading answers the offloaded requests with the edge exit, the geometric ensemble with the geometric mean of both exits, and DriftGate with its combination, whose weight uses only offloaded requests. Curves interpolate the seed means between thresholds. The dotted line marks the offloading ratio of confidence-based offloading at 0.8 nats.

TABLE 8

Accuracy (%) of DriftGate and variants that change one design choice, over the first day and in the frozen replay. “Correction” is the prior correction of Eq. (6) with $r = 1/2$. The fixed weight of 0.2 was chosen on development runs with the CNN.

Variant	First day					Frozen replay	
	S1	S2	Random mob.	CIFAR-100	ResNet-18	S1	S2
DriftGate (correction, adaptive weight)	68.31	71.70	71.09	41.29	66.64	71.95	76.54
No correction, adaptive weight	67.11	69.29	69.33	37.50	66.46	72.46	75.88
No correction, $w = 0.5$ (Probability average)	67.32	69.88	69.58	38.15	66.39	72.27	75.90
Correction, $w = 0.5$	68.13	71.77	70.96	41.45	66.71	71.53	76.23
Correction, $w = 0.2$	68.16	71.03	70.51	40.95	64.02	72.58	76.66
Corrected edge exit only	67.87	70.49	69.89	40.69	61.72	72.74	76.59
Correction, learned weight	67.88	70.66	70.64	40.55	64.36	71.71	75.26
Correction, product	67.90	71.76	70.78	41.73	66.59	71.12	76.00

TABLE 9

Accuracy (%) over the first day as the number of devices grows, with 5, 20, and 50 cells, for confidence-based offloading (Conf.), the probability average (Avg.), and DriftGate (DG). Gain is DriftGate’s per-seed gain over the strongest alternative. The last two columns give the offloading ratio of confidence-based offloading and the offloading ratio at which DriftGate first matches its accuracy.

Devices	Conf.	Avg.	DriftGate	Gain	Offloading	
					Conf.	DG
50	62.87	67.32	68.31	0.95	0.920	0.000
200	65.26	68.90	69.00	0.10	0.949	0.223
500	67.03	70.38	70.11	−0.27	0.955	0.311

of Bayes’ rule.

6.7 Scale and the Least Accurate Devices

Does the gain hold in larger deployments? Table 9 repeats the S1 layout with 200 and 500 devices. The number of devices per cell (about 11.5), the share of Main requests (about 65%), and a_k (about 0.25) stay the same, but the edge exit becomes more accurate (62.9%, 65.3%, and 67.1% alone). DriftGate stays 3.1 to 5.4 points above confidence-based offloading, but its margin over the strongest alternative shrinks from 0.95 points with 50 devices to 0.10 points with 200 devices and becomes −0.27 points with 500 devices, where the probability average

TABLE 10

Accuracy (%) of the 10% least accurate devices over the first day. The devices are selected anew for each rule and each evaluation round.

Setting	Conf.-based	Prob. average	Logit sum	DriftGate
S1	40.98	46.62	46.38	48.00
S2	39.53	45.89	45.88	47.43
S1-fast	41.61	47.99	47.66	49.37
Partial participation	31.28	39.27	39.67	43.24

is the strongest rule. With 200 and 500 devices, the device exit alone is less accurate than confidence-based offloading, so DriftGate needs some offloading to match it. With 500 devices, DriftGate matches the accuracy of confidence-based offloading while offloading 31% of the requests instead of 95.5%. These results describe deployments with the cell count scaled with the number of devices. They do not measure the throughput of a single edge server.

Does the average gain hide devices that do worse? Table 10 reports the accuracy of the 10% least accurate devices in each evaluation round. Over the first day, DriftGate raises this accuracy by 7.0 to 12.0 points over confidence-based offloading and by 1.4 to 3.6 points over the strongest combining rule. Because the devices are selected anew for each rule and round, this result describes the lower tail of the accuracy distribution,

TABLE 11

Per-request cost of the inference path for the CNN. Red values are assumptions to be replaced by measurements.

Component	Smartphone	Jetson
Device block and device exit (ms)	11.8	2.9
Device block and device exit (mJ)	TBD	TBD
Uplink payload	32 KB	32 KB
Uplink time at 50 Mbps (ms)	5.2	5.2
Edge block, exit, and correction (ms)	0.6	0.6
Downlink payload per edge call	44 B	44 B
Combination on the device (μ s)	5	3

not every individual device.

6.8 Cost on Mobile Devices

[TODO: Replace this subsection with measured values. The values below are the inputs of a cost model, not measurements; red numbers are assumptions.]

Setup. We measure the inference path on two smartphones, [TODO: Samsung Galaxy model 1] and [TODO: model 2], and on an NVIDIA Jetson [TODO: model] board. The device block and the device exit run with [TODO: runtime, e.g., PyTorch Mobile or TensorRT]. The edge server is a desktop with an RTX 4090 GPU. We measure latency [TODO: over N requests after warm-up] and energy with [TODO: power monitor or Android battery API]. The network is [TODO: Wi-Fi / LTE], with an uplink of [TODO: measured] Mbps and a round-trip time of [TODO: measured] ms.

Per-request cost. Table 11 lists the cost of each step. The device-side cost of DriftGate beyond confidence-based offloading is the combination and a downlink of 44 bytes per edge call instead of one label.

End-to-end latency and energy. If each branch has a fixed cost, the mean latency of a request is $(1 - \beta)$ times the device-side time plus β times the time of an offloaded request, which adds the uplink, the round trip, and the edge time. With the assumed values of Table 11 and a round-trip time of 20 ms, this model gives **35.5 ms** per request for confidence-based offloading in S1 at an offloading ratio of 0.92 (62.9% accuracy over the first day), **37.6 ms** for DriftGate with every request offloaded (68.3%), and **24.7 ms** for DriftGate with the online controller at an offloading ratio of 0.50 (67.7%). Radio tail energy, idle power, and request spacing can make measured energy differ from such a linear model. [TODO: Report measured latency (mean and 95th percentile) and energy per request on a replayed request stream for the operating points above, and plot accuracy against latency and against energy.]

7 Discussion

What remains for devices away from home. DriftGate gains most where the device exit is strong and gives up some accuracy where the edge exit is strong. A rule that knew the kind of every request, and answered Main requests with the device exit and all other requests with the edge exit, would reach 72.9% in S1 over the first day, compared with 68.3% for DriftGate. Most of this gap lies in the OOP and OOR requests of devices away from home. Closing it requires a signal that recognizes, for a single request, that its class is new to the

device. The confidence of the device exit gives only a weak signal (Section 2.3), and the label-free features we combined with a logistic regression did not close the gap in our runs. Distances in the feature space of the device block, measured against the device’s own training data, are a natural next candidate.

Trained models. The first-day results of Table 4 include the early phase of training, when the edge exit is weak and the device exit carries more of the accuracy. With trained models (Section 6.3), the gap between the two exits and the other combining rules narrows, and the parts of DriftGate that help depend on the scenario. In the S1 replay, the corrected edge exit alone and a fixed weight of 0.2 are more accurate than DriftGate, and so is the uncorrected probability average. In the S2 replay, DriftGate remains the most accurate rule. A deployment that serves trained models over many days should therefore check the combination and the weight on its own traffic, for example on a development day. Our replay reuses the same day of movement for S2’s real traces, so it is not an evaluation on a new day.

Relation to training. DriftGate acts after training and only on the answers of the requesting device, and it does not change the shared model. The location-aware training policy of Section 2.4 showed a gain for devices at home together with a loss for devices away, and we have not tested whether combining it with DriftGate keeps both effects. Because DriftGate needs only the two output vectors, the class list of the device, and the class counts of the cell, it can run on top of other ways of training the two exits, such as different personalization ratios, personalized federated learning objectives, or the online distillation of Kim et al. [23]; we have not measured its accuracy with these.

Information at the edge. The prior correction uses the device’s class list M_k and the class counts of the cell. In split learning, the edge receives training labels and already holds both. In variants that keep labels on the device, the device can send its class list once, which reveals no more than which classes it trains on.

Scope. We studied changes in which classes a device sees, which is how mobility changes the traffic of a recognition application in our simulation. Changes in how the same classes look, such as lighting or sensor differences between places, affect both exits and are outside the scope of the correction. The class mix of the requests follows a simulation setting (ρ), and the real GeoLife traces determine only where devices are.

8 Conclusion

When users move, the requests of a mobile device alternate between classes that its personalized exit has learned and classes that only the shared edge exit knows. Choosing one exit by confidence follows this change poorly, because the personalized exit is confident on many classes it has never seen. DriftGate combines the two exits instead. It averages their class probabilities after correcting the edge exit’s bias against the device’s own classes, sets the weight of each exit from the device’s recent offloaded requests, and offloads only uncertain requests. Over the first simulated day, DriftGate has the highest mean accuracy of eleven inference rules in eight settings, and in five of them it exceeds every rule that uses the edge for all requests while offloading at most half

of the requests. Much of this lead comes from the early phase of training. With trained models, DriftGate is the most accurate rule on GeoLife traces and 0.3 points below a plain probability average in the commute scenario, so the benefit of its correction and weighting depends on the deployment. [TODO: One sentence on the measured latency and energy on smartphones and Jetson boards.]

References

- [1] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang, "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in Proc. ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), 2017.
- [2] S. Laskaridis, S. I. Venieris, M. Almeida, I. Leontiadis, and N. D. Lane, "SPINN: Synergistic progressive inference of neural networks over device and cloud," in Proc. ACM International Conference on Mobile Computing and Networking (MobiCom), 2020.
- [3] O. Gupta and R. Raskar, "Distributed learning of deep neural network over multiple agents," *Journal of Network and Computer Applications*, vol. 116, pp. 1–8, 2018.
- [4] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, "Split learning for health: Distributed deep learning without sharing raw patient data," *arXiv preprint arXiv:1812.00564*, 2018.
- [5] C. Thapa, P. Chamikara, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in Proc. AAAI Conference on Artificial Intelligence, 2022.
- [6] D.-J. Han, D.-Y. Kim, M. Choi, C. G. Brinton, and J. Moon, "SplitGP: Achieving both generalization and personalization in federated learning," in Proc. IEEE Conference on Computer Communications (INFOCOM), 2023.
- [7] A. Rizwan, D.-J. Han, M. F. Pervej, C. G. Brinton, A. F. Molisch, and M. Choi, "Efficient split learning with overlapping areas: Handling distribution shift in multi-cell networks," *IEEE Transactions on Networking*, vol. 34, pp. 2834–2849, 2026.
- [8] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in Proc. International Conference on Pattern Recognition (ICPR), 2016.
- [9] Y. Deng, M. M. Kamani, and M. Mahdavi, "Adaptive personalized federated learning," *arXiv preprint arXiv:2003.13461*, 2020.
- [10] Y. Zheng, X. Xie, and W.-Y. Ma, "GeoLife: A collaborative social networking service among user, location and trajectory," *IEEE Data Engineering Bulletin*, vol. 33, no. 2, pp. 32–39, 2010.
- [11] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [12] H.-Y. Chen and W.-L. Chao, "On bridging generic and personalized federated learning for image classification," in Proc. International Conference on Learning Representations (ICLR), 2022.
- [13] L. Jiang and T. Lin, "Test-time robust personalization for federated learning," in Proc. International Conference on Learning Representations (ICLR), 2023.
- [14] M. Wołczyk, B. Wójcik, K. Bałazy, I. Podolak, J. Tabor, M. Śmieja, and T. Trzciński, "Zero time waste: Recycling predictions in early exit neural networks," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2021.
- [15] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002.
- [16] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in Proc. International Conference on Learning Representations (ICLR), 2017.
- [17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. International Conference on Machine Learning (ICML), 2017.
- [18] Y. Mao, C. You, J. Zhang, K. Huang, and K. B. Letaief, "A survey on mobile edge computing: The communication perspective," *IEEE Communications Surveys and Tutorials*, vol. 19, no. 4, pp. 2322–2358, 2017.
- [19] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-deep networks: Understanding and mitigating network overthinking," in Proc. International Conference on Machine Learning (ICML), 2019.
- [20] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in Proc. International Conference on Machine Learning (ICML), 2021.
- [21] C. T. Dinh, N. Tran, and J. Nguyen, "Personalized federated learning with Moreau envelopes," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2020.
- [22] L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "Exploiting shared representations for personalized federated learning," in Proc. International Conference on Machine Learning (ICML), 2021.
- [23] M. Kim, H. Park, and M. Kwon, "Personalized split federated learning with early exit: Pretraining and online learning against label shifts," *IEEE Internet of Things Journal*, vol. 12, no. 22, pp. 47 069–47 082, 2025.
- [24] Z. C. Lipton, Y.-X. Wang, and A. J. Smola, "Detecting and correcting for label shift with black box predictors," in Proc. International Conference on Machine Learning (ICML), 2018.
- [25] A. Alexandari, A. Kundaje, and A. Shrikumar, "Maximum likelihood with bias-corrected calibration is hard-to-beat at label shift adaptation," in Proc. International Conference on Machine Learning (ICML), 2020.
- [26] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar, "Long-tail learning via logit adjustment," in Proc. International Conference on Learning Representations (ICLR), 2021.
- [27] C. Genest and J. V. Zidek, "Combining probability distributions: A critique and an annotated bibliography," *Statistical Science*, vol. 1, no. 1, pp. 114–135, 1986.
- [28] B. Liang and Z. J. Haas, "Predictive distance-based mobility management for PCS networks," in Proc. IEEE Conference on Computer Communications (INFOCOM), 1999.
- [29] A. Krizhevsky, "Learning multiple layers of features from tiny images," University of Toronto, Tech. Rep., 2009.